School of Computing, Mediterranean College
Ioannis Tsioulis, i.tsioulis@mc-class.gr

Sui Move Thesis Proposals

Overview of Sui

Sui is a Layer 1 blockchain designed for high throughput, low latency and low transaction costs. Instead of an account-based design, Sui uses an object-centric data model in which every asset is an object with its own identifier, owner and type, which allows independent transactions to be processed in parallel. Smart contracts on Sui are written in Move, a programming language originally designed for the Diem blockchain, whose type system prevents digital assets from being copied or accidentally destroyed. Programmable Transaction Blocks allow many operations to be combined into a single transaction, and zkLogin allows users to access applications with familiar web credentials. Developers can access resources through the Sui Developer Portal, and the ecosystem is supported by an active developer community and grant programmes.

Links and Repositories

Official site	https://sui.io/
Sui GitHub repository	https://github.com/MystenLabs/sui
Sui developer tooling	https://docs.sui.io/tooling
The Move Book	https://move-book.com/
Electric Capital crypto-ecosystems repo	https://github.com/electric-capital/crypto-ecosystems
Move issue classifier GitHub repository	https://github.com/Vbakalis/move-issue-classifier
Move issue dataset on Zenodo	https://doi.org/10.5281/zenodo.19682589

About the Move Issue Classifier

Within the School of Computing at Mediterranean College, a dataset of 3,321 Move smart-contract snippets from the Sui ecosystem has been built and published on Zenodo, with each snippet labelled as Perfect, SecurityError, SemanticError, StyleError or SyntaxError. Several approaches have been compared on this dataset: TF-IDF with classical classifiers, full fine-tuning of CodeBERT, CodeBERT with LoRA adapters, and a 4-bit LoRA fine-tune of the 7B Qwen2.5-Coder decoder. CodeBERT with LoRA achieved about 99% accuracy while training less than 1% of the model parameters. The model is integrated into NiceMove, a VS Code extension that classifies Move code locally and can request a fix suggestion from Claude when an issue is detected. The dataset will be extended with new data through the data-collection thesis topics described in this document.

NiceMove VS Code extension: https://marketplace.visualstudio.com/items?itemName=BakalisVasilis.nicemove

These high scores open an important research question: to what extent do they reflect genuine recognition of Move issues? The error samples were created by taking valid code and introducing a single issue, a process that can leave systematic patterns which a model may learn instead of the issue itself. Variants derived from the same original snippet may appear in both the training and the test data, since duplicate removal targets exact copies, and the classes differ in snippet length and in their typical keywords, to the point that even a simple TF-IDF model reaches 97%. Further important

directions are evaluation over multiple random seeds and testing on real issues from real projects. This topic aims to answer this question and to provide a robust and reproducible evaluation of the models.

This document proposes one research topic on the evaluation of the classifier and one data-collection topic for each of the five categories. Each topic includes a **theoretical component** (analysis, study, comparison) and a **practical component**. Communication with the supervisor for questions and ongoing collaboration with the Sui ecosystem are included.

Each topic is offered to one student only. Topics marked ALLOCATED have been assigned and are no longer available.

Research Topic: Evaluation of the Move Issue Classifier ALLOCATED

Beyond Injected Errors: Reproducing and Stress-Testing LLM-Based Issue Classification for Sui Move Smart Contracts

This topic addresses a central question: when a model scores 99% on Move issue classification, how much of that score comes from truly recognising the issue, and how much from shortcuts created by the way the dataset was built? The student will first reproduce the existing results, then test the models under stricter and more realistic conditions, and finally repeat the whole evaluation with the new data collected by the group. The result will be a reliable and reproducible picture of what these models can do, and a stronger benchmark for anyone working on AI tools for Move.

Theoretical component

The student studies how pre-trained code models such as CodeBERT work and how parameter-efficient fine-tuning with LoRA adapts them, and why models trained on synthetic or injected errors often learn surface patterns instead of the intended concept (shortcut learning). The study covers data leakage and code duplication in machine-learning datasets for code, the known gap between results on synthetic vulnerability benchmarks and on real vulnerabilities, and the specific features of Move and the Sui object model (abilities, ownership, linear resources) that make its issues different from those of Solidity. The student also examines what the Move compiler can already detect on its own, in order to explain where an AI classifier adds value beyond existing tools.

Practical component

The student starts by reproducing the existing work. Using the code in the classifier repository and the dataset on Zenodo, the student reruns the TF-IDF, CodeBERT and CodeBERT with LoRA experiments and confirms that the reported results can be obtained again. According to the repository documentation, the TF-IDF and CodeBERT experiments do not require Apple MLX, while the Qwen experiments use MLX and therefore need an Apple Silicon computer. This step teaches the student the codebase and ensures that all later results are comparable. The student then checks the labels with the Sui Move compiler wherever a snippet can be compiled on its own: according to the class definitions, SyntaxError samples should fail to parse, SemanticError samples should fail type checking, and the remaining classes should compile. The share of samples that disagree with their label is reported per class.

Next, the student tests whether the models rely on shortcuts. Snippets that are near-copies of each other, for example variants of the same original function with a different inserted error, are detected by code similarity and grouped into families. Two new data splits are created and saved as fixed files: one in which all members of a family are kept together in either the training or the test set, and one in which all code from the same repository is kept together, so that the test set contains only projects the model has never seen. The student also trains simple reference models

that see only the length of a snippet or only its formatting, and re-evaluates the main models after formatting all code in the same way, renaming identifiers to neutral names, and hiding the keywords that are most typical of each class. All experiments are repeated over several random seeds and reported as mean and standard deviation with confidence intervals. Every experiment is written as a script, so that the full evaluation can be rerun with a single command.

The student then builds a small test set of real, naturally occurring issues. Real problems are collected from public security audit reports of Sui projects and from bug-fix commits in Sui repositories listed in the Electric Capital repository, keeping the code before the fix as the issue and the code after the fix as the corrected version. These samples are labelled with the same class definitions, together with a second annotator, and agreement is measured. All models, trained on the existing dataset, are evaluated on these real samples, which shows whether performance holds outside the synthetic setting.

Finally, the student adds the new data collected through the data-collection thesis topics and reruns the complete pipeline: label checks, family and repository splits, shortcut tests, all models and all seeds, and the evaluation on real issues. This shows whether a larger dataset reduces the shortcuts and improves real-world performance.

Main outcomes and further work

The main outcomes of this topic are the reproduction, the label-consistency check, the results under the new splits and shortcut tests over multiple seeds, the evaluation on real issues, and the updated results with the new data. The dissertation can be extended further with an analysis of results by issue type on the real samples, a split in which some types of inserted errors appear only in the test set, a first experiment with hierarchical classification using the fine-grained sublabels already present in the dataset, and a full literature review.

Required skills: Python, scikit-learn, basic PyTorch and Hugging Face, and basic statistics. Familiarity with Move is helpful and can be developed during the project with support from the group.

Data-Collection Thesis Topics

The following topics extend the Move issue dataset. Each topic includes a **theoretical component** (analysis, study, comparison) that the student explores in depth, and a **practical component** in which the student collects and annotates Move code snippets from open-source Sui repositories listed in the Electric Capital crypto-ecosystems repository. More experienced Move developers are encouraged to choose the Semantic Error and Security Error topics, while newer developers may prefer the Perfect, Syntax Error and Style Error topics.

Each collected sample is recorded with the following fields: **Code** (the snippet), **Label** (the category), **Sublabel** (the specific type of issue, for example MissingDelimiter or UseAfterMove), **Output/Fix** (the corrected code and a short explanation, empty for Perfect), and **Source** (repository link, file path and commit). Recording the source is essential, because it allows the dataset to be split correctly and checked in the future.

To keep the new data useful for research, all students follow the same quality rules. Snippets are taken from different files and functions, and no two error samples are created from the same original snippet. The original formatting, comments and identifier names are kept unchanged, and the only modification is the inserted issue. Each student uses a wide variety of issue types within the category instead of repeating the same kind of change, and avoids making error samples systematically shorter

or longer than typical code. Where possible, students also look for real issues in commit histories and public audit reports, and mark these samples as real rather than inserted. New samples are checked against the existing public dataset to avoid duplicates, and only code from repositories with open-source licences is used. Data that meet these requirements will be considered for inclusion in future public releases of the dataset.

Category: Perfect

Idiomatic Design Patterns in Sui Move: Capabilities, Witnesses and Object Ownership in Real-World Smart Contracts

Well-written Sui Move code relies on a small number of recurring design patterns, such as capabilities for access control, the one-time witness for creating unique types, hot potato objects that must be consumed within the same transaction, and the careful use of owned, shared and wrapped objects. The theoretical component studies these patterns, the Move abilities (copy, drop, store and key) that make them possible, and how they help developers write correct and secure contracts. The practical component collects correct, idiomatic snippets from real Sui projects, confirms that each one compiles without errors, and records which design pattern each snippet demonstrates, producing a well-documented set of high-quality examples.

Code samples per student: 400

Category: Syntax Error

AI-Driven Syntax Checking in Move Smart Contracts: Comparing Language Models with the Move Compiler

The Move compiler detects syntax errors reliably, but its messages are not always easy for new developers to understand. The theoretical component studies how the Move parser works, the most common categories of syntax errors, the syntax changes introduced by the Move 2024 edition, and how language models can complement the compiler by explaining an error and suggesting a fix. The practical component collects a varied set of syntax errors, covering for example malformed declarations, unbalanced delimiters, missing separators and invalid use of keywords, and for each sample records the compiler message together with the correct fix. The student then compares the compiler messages with the explanations produced by language models for a subset of the samples.

Code samples per student: 800

Category: Semantic Error

Use of LLMs for Semantic Understanding in Move Code

Semantic errors in Move arise when code is syntactically valid but breaks the rules of the type system and the Sui object model, for example missing abilities, use of a value after it has been moved, borrowing violations, type mismatches, or objects that do not follow the rules required by Sui. The theoretical component studies these rules and how the Move compiler enforces them, and examines how well language models understand them compared with the checks performed by the compiler. The practical component collects a varied set of semantic errors across these categories, records the compiler message and the correct fix for each sample, and analyses which kinds of semantic errors are the most difficult for language models to recognise.

Code samples per student: 800

Category: Security Error ALLOCATED

Security Models and Common Vulnerabilities in Sui Move Smart Contracts

The design of Move removes several vulnerability classes that are common in other smart-contract languages; for example, the absence of dynamic dispatch prevents classic re-entrancy attacks. Security problems still occur in Sui Move contracts, but they take different forms, such as missing or incorrect capability checks, public functions that expose sensitive operations, incorrect handling of shared objects, precision and rounding errors in financial calculations, and reliance on unverified external data. The theoretical component builds a structured taxonomy of these vulnerabilities, compares it with established smart-contract vulnerability classifications, and explains how the Sui object model changes the security picture. The practical component collects security issues that compile successfully, labels each sample with its vulnerability type from the taxonomy, and gives a clear explanation of the risk and the fix. Because realistic security samples require careful analysis, this topic has a smaller sample target and places particular value on real issues found in public audit reports.

Code samples per student: 600

Category: Style Error

Theoretical Code Style Guidelines in Sui Move

Consistent style makes smart contracts easier to review, audit and maintain. The theoretical component studies the coding conventions described in the Move Book and the Sui documentation, including naming of modules, structs, functions and error constants, documentation comments, module organisation and the use of named constants instead of magic numbers, and examines how formatting tools such as the Prettier Move plugin support these conventions. The practical component collects functionally correct code with a wide variety of style violations, records the specific convention that each sample breaks together with the corrected version, and summarises the guidelines into a practical checklist for Sui Move developers. Since the existing dataset already contains a large number of style samples, the emphasis is on covering conventions that are not yet well represented.

Code samples per student: 600

Collaboration

The topics in this document are complementary. The students of the data-collection topics provide the new data, and the student of the research topic adds these data to the evaluation; a data-collection student can also act as the second annotator for the real-issue test set. Each dissertation is written individually and covers the student's own contribution.

Appendix: Categories and Sublabels

Every sample has one Label and, except for Perfect, one Sublabel taken from the lists below. The Sublabel is chosen from the closed list given for its category and is spelled exactly as given, so that all collected data use identical sublabel names. Perfect samples have an empty Sublabel. One optional field, **Notes**, is required only when the Sublabel is Other, and states in one sentence what the problem is.

SyntaxError samples take one of the following sublabels: MissingDelimiter, InvalidKeyword, MalformedDeclaration, MalformedExpression, ImportSyntax, GenericsOrAbilitySyntax, Other.

SemanticError samples take one of the following sublabels: TypeMismatch, UseAfterMove, BorrowViolation, UnusedResource, MissingAbility, WrongVisibility, UnreachableCode, ObjectOwnership, HotPotatoMisuse, SharedObjectMisuse, Other.

StyleError samples take one of the following sublabels: MagicNumber, UnclearErrorCode, Spacing, DeadCode, MissingDocumentation, IncorrectIndentation, InconsistentNaming, FunctionTooComplex, InconsistentFunctionSignature, DeepNesting, InconsistentImportOrdering, Deprecated, Other.

SecurityError samples take one of the following sublabels: AccessControl, Reentrancy, ArithmeticOverflow, DivisionByZero, InputValidation, OraclePriceManipulation, FlashLoanLiquidation, Governance, Bridge, RandomnessCrypto, Frontrunning, DoSUnbounded, TimeDependency, ResourceHandling, ErrorHandling, Other.

Perfect samples have no sublabel.

The following rules apply when a sublabel is assigned. Each snippet should illustrate a single issue, and when two issues are present the most prominent one is the Sublabel. Other is used only for issues that fit no listed sublabel, and the snippet is first checked against every listed sublabel of its category. Code that the compiler rejects or flags as unreachable is labelled UnreachableCode (SemanticError), while code that compiles normally but is unused is labelled DeadCode (StyleError). An error code that is only hard to read is labelled UnclearErrorCode (StyleError), while a silently ignored failure or an ignored result is labelled ErrorHandling (SecurityError).